SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:50**

Annexure A

SCENARIO BASED

Code of Conduct:

I Hari Vishnu N / 192524319 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. CPU Scheduling in a Real-Time Ticket Booking System :-

(1) Suitable Scheduling Algorithm :-

Priority Scheduling (Preemptive)

(2) Justification :-

- High-priority payment transaction (P_1) are executed immediately.
- Reduces waiting time for critical operations.
- Improves CPU utilization by serving important tasks first, while medium and low priority tasks run when CPU is free.

(3) Sample Executive Order

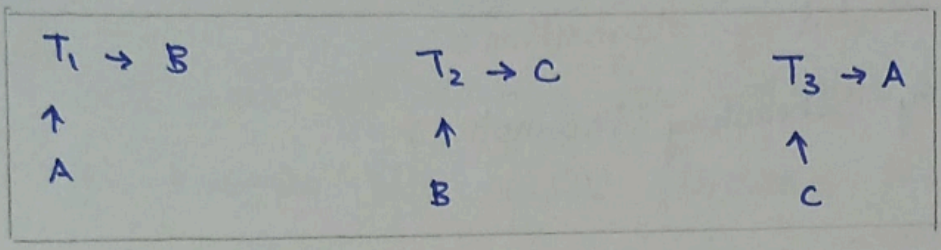
P_1 (Payment) → P_2 (Seat Availability) → P_3 (Ticket Confirmation & Printing)

(4) Drawback

- STARVATION: Low-priority processes (P_3) may wait indefinitely if high-priority processes continuously arrive.

Q2. DEADLOCK IN A BANKING TRANSACTION ERROR :

(1) Text-based Resource Allocation Graph (RAG)



T_1 holds A → requests B

T_2 holds B → requests C

T_3 holds C → requests A

(2) Yes, deadlock.

→ A circular wait exists: $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$

→ None of the transactions can proceed.

(3) Recommend Technique :

Deadlocks Prevention -

- Enforce a fixed resource acquisition order (A → B → C)
- Elimination circular wait.

(4) Redesign :

All transactions must lock resources in the same order.

Lock A
Lock B
Lock C

Perform Task

Release C
Release B
Release A

Q3. MOBILE OS APP SWITCHING LAG:

(1) Why CPU shows High Idle Time

- Most apps are waiting for I/O (disk / network / user input).
- CPU remains idle while processes are blocked.

(2) Is 20 ms Quantum Good? - No

- With 20 inactive apps, 20 ms is relatively large.
- Apps wait longer before getting CPU, causing lag.

(3) Recommendation

Use Round Robin with a smaller quantum (5-10 ms) or Multilevel Feedback Queue (MLFQ).

(4) Benefits

- Faster response time
- Reduced app-switching delay.
- Better throughput and smoother multitasking

Q4. DEADLOCK RISK IN WINDOWS OS UPDATE

Deadlock can occur.

Circular Wait:

M_1 holds R_1 → waits for R_2

M_2 holds R_2 → waits for R_3

M_3 holds R_3 → waits for R_1

Cycle: $M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow M_1$

Banker's Algorithm Approach:

- Grant resources only if the system remains in a safe state.
- Delay requests that would create an unsafe allocation.

Prioritization:

Execute modules in dependency order:

$M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow M_4$

Q5. Cloud Server CPU utilization analysis.

Given: • $p = 0.8$ • $n = 7$

Formula: $U = 1 - p^n$

Calculation:

CPU Utilization: $U = 1 - (0.8)^7$

$= 1 - 0.2097$

$= 0.7903$

Interpretation:

79% utilization indicates the server is finally well utilized though there is some idle capacity.

Effect of Changing Number of VMs

- INCREASE VMs: Better CPU utilization and throughput (up to a limit)
- TOO MANY VMs: More context switching and resources contention, reducing perfoamance.
- DECREASE VMs: Lower throughput and more CPU idle time.

Two OS-Level Strategies:

- Use efficient CPU scheduling.
- Overlap I/O & CPU using asynchronus I/O or improve load balancing across VMs to keep the CPU busy.